

REST - Country Web Service:

Country.java:

```
package com.cognizant.spring_learn.model;

public class Country {

    private String code;

    private String name;

    // No-arg constructor required for instantiation frameworks
    public Country() {}

    public Country(String code, String name) {

        this.code = code;

        this.name = name;

    }

    // Getters and Setters (Crucial for Jackson Serialization)
    public String getCode() { return code; }

    public void setCode(String code) { this.code = code; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    @Override
```

```

public String toString() {
    return "Country [code=" + code + ", name=" + name + "]\n";
}
}

```

date-format.xml:

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

    <bean id="india" class="com.cognizant.spring_learn.model.Country">
        <property name="code" value="IN" />
        <property name="name" value="India" />
    </bean>

</beans>

```

CountryController.java:

```

package com.cognizant.spring_learn.controller;

import org.slf4j.Logger;

import org.slf4j.LoggerFactory;

```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.model.Country;
```

```
@RestController
```

```
public class CountryController {
```

```
    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
```

```
    // Injecting the underlying application context to fetch XML-defined beans
```

```
    @Autowired
```

```
    private ApplicationContext context;
```

```
    @RequestMapping(value = "/country", method = RequestMethod.GET)
```

```
    public Country getCountryIndia() {
```

```
        LOGGER.info("--- START: Entering getCountryIndia() REST endpoint ---");
```

```
        // Fetching the pre-configured bean from the XML context
```

```
        Country india = context.getBean("india", Country.class);
```

```
        LOGGER.info("Successfully fetched bean: {}", india);
```

```
        LOGGER.info("--- END: Exiting getCountryIndia() REST endpoint ---");
```

```
        return india;
```

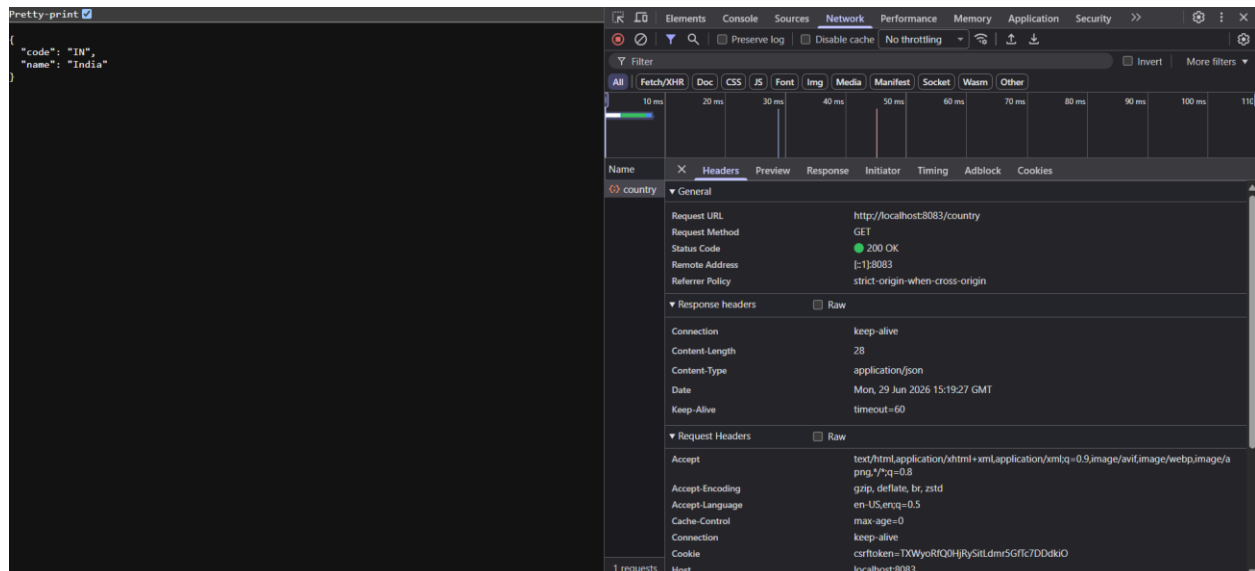
```
    }
```

```
}
```

2. SME Technical Explanation

Aspect 1: What happens in the controller method?

1. **Request Interception:** The client fires an HTTP GET request to `http://localhost:8083/country`. Spring's front controller, the `DispatcherServlet`, intercepts the request and checks its handler mappings.
2. **Method Routing:** It matches the `/country` path and GET method to `CountryController.getCountryIndia()`.
3. **Execution Context:** Once execution shifts inside the method, your application triggers the SLF4J logger to log the entry event.
4. **IoC Lookup:** The method makes an explicit pull call (`context.getBean("india", Country.class)`) to the injected Spring Application Context. The framework doesn't instantiate a new object here; it fetches the single, shared instance (singleton scope by default) built during application boot from your XML map definitions.
5. **Return Pipeline:** The method finishes by handing off the raw `Country` Java object back to Spring's processing pipeline.



▼ Request Headers ☐ Raw	
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.5
Cache-Control	max-age=0
Connection	keep-alive
Cookie	csrftoken=TXWyoRfQ0HjRySitLdmr5GfTc7DDdkio
Host	localhost:8083
Sec-Ch-Ua	"Brave";v="149", "Chromium";v="149", "Not)A;Brand";v="24"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Sec-Gpc	1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36


http://localhost:8083/country
Save
Share

GET

Send

Docs
Params
Authorization
Headers 6
Body
Scripts
Tests
Settings
Cookies

Headers

Hide auto-generated headers

Key	Value	Description	Bulk Edit	Presets	
✓ Postman-Token	<calculated when request is sent>				
✓ Host	<calculated when request is sent>				
✓ User-Agent	PostmanRuntime/7.54.0				
✓ Accept	*/*				
✓ Accept-Encoding	gzip, deflate, br				
✓ Connection	keep-alive				
Key	Value	Description			

Body

200 OK • 5 ms • 184 B •

JSON

Preview

Visualize

```

1  {
2    "code": "IN",
3    "name": "India"
4  }

```